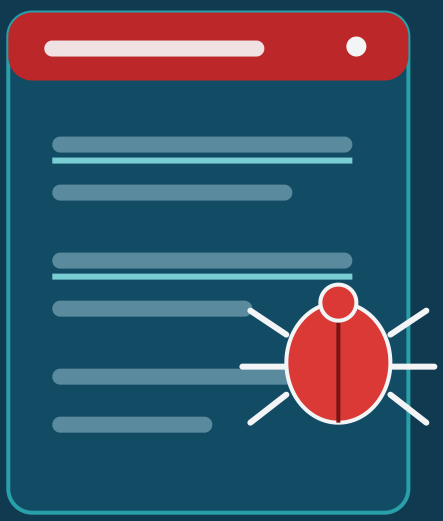
GESTIÓN DE CALIDAD DE SOFTWARE

Guía de redacción de bug reports

Principios, redacción campo por campo, severidad y prioridad, evidencias y ejemplos resueltos con estructura estilo Qase



Elaborado por: Pardo Pozo Juan Diego
Carrera de Ingeniería de Sistemas · UMSS
Versión 1.0 · Octubre 2026

CONTENIDO

Índice

1. Qué es un bug report

1.1 Definición y propósito

1.2 Error, defecto, falla y bug

1.3 El reporte como acto de comunicación

2. Ciclo de vida de un defecto

3. Principios de un buen reporte

3.1 Reproducibilidad

3.2 Un defecto por reporte

3.3 Objetividad

3.4 Evidencia

3.5 Trazabilidad

3.6 Accionabilidad

4. Anatomía de la plantilla

5. Redacción campo por campo

5.1 ID y Estado

5.2 Prioridad y Severidad

5.3 Título

5.4 HU y C.A. violado

5.5 T.C. de origen

5.6 Reproducibilidad

5.7 Ambiente

5.8 Descripción

5.9 Precondiciones

5.10 Pasos para reproducir

5.11 Datos de entrada

5.12 Resultado actual

5.13 Resultado esperado

5.14 Evidencias

5.15 Reportado por

6. Severidad y prioridad en profundidad

7. Cómo construir los pasos de reproducción

8. Clasificación de defectos por tipo

9. Evidencias efectivas

10. Ejemplos completos resueltos

11. Lenguaje: palabras y estilo

12. Antipatrones frecuentes

13. Lista de verificación final

14. Glosario

Cómo leer esta guía

Los recuadros REGLA son obligatorios: si se incumplen, el reporte se considera mal redactado. Los recuadros CORRECTO e INCORRECTO comparan redacciones reales. Los recuadros ATENCIÓN señalan errores sutiles que suelen pasar una revisión rápida. Los ejemplos combinan defectos reales encontrados en UMSSPIRA (el sistema de la plataforma de mentorías de egresados, historia de usuario HU-6.2 "Configurar áreas técnicas de especialidad") con casos genéricos de un sistema ficticio de comercio electrónico.

Reglas base del reporte

Estas diez reglas resumen la guía. Cada capítulo las desarrolla y añade buenas prácticas de la industria. Un reporte que las cumple todas es un reporte accionable.

#	Regla	Consecuencia práctica
1	Un reporte describe **un solo defecto**.	Cada defecto se prioriza, asigna y cierra por separado.
2	El defecto debe ser **reproducible** paso a paso.	Quien lo lee llega al mismo fallo sin preguntar nada.

3	Se describen **hechos observables**, no opiniones ni causas supuestas.	"Muestra el mensaje X", no "el guardado está mal hecho".
4	El **título** dice qué falla, dónde y bajo qué condición.	Se entiende el defecto sin abrir el detalle.
5	**Severidad** y **prioridad** se asignan por separado.	Severidad es impacto técnico; prioridad es urgencia de negocio.
6	Resultado actual y resultado esperado van **enfrentados**.	La diferencia entre ambos es el defecto.
7	Toda afirmación se respalda con **evidencia**.	Captura, video, log, respuesta HTTP o consulta.
8	El reporte cita la **HU**, el **C.A.** y el **caso de prueba** de origen.	Permite medir cobertura y auditar el defecto.
9	El **ambiente** queda identificado por completo.	Versión, navegador, URL, resolución, datos.
10	Si un campo no aplica, la celda queda **vacía**.	No se escribe "N/A": genera ruido visual.

CAPÍTULO 1

1. Qué es un bug report

1.1 Definición y propósito

Un **bug report** (reporte de defecto) es el documento que registra, de forma estructurada y reproducible, una discrepancia entre el comportamiento observado de un sistema y su comportamiento esperado. No es una queja ni una nota suelta: es un artefacto de ingeniería cuyo objetivo es que el equipo de desarrollo pueda **localizar, reproducir, corregir y verificar** el defecto con el menor esfuerzo posible.

Un buen reporte responde seis preguntas sin que el lector tenga que buscar a quien lo escribió: qué ocurre, dónde ocurre, en qué condiciones, qué debería ocurrir en su lugar, con qué frecuencia se repite y con qué evidencia se demuestra. Si falta cualquiera de esas respuestas, el reporte genera idas y vueltas que retrasan la corrección.

Qué es

- Una especificación reproducible de un fallo.
- Una evidencia trazable hacia un criterio de aceptación.
- Un insumo accionable para desarrollo.

Qué no es

- Una opinión sobre la calidad del código.
- Un diagnóstico de la causa raíz.
- Un reclamo dirigido a una persona.

1.2 Error, defecto, falla y bug

El vocabulario importa porque cada término señala un momento distinto en la cadena causal. Usarlos con precisión evita confusiones en las reuniones de triaje y en la documentación.

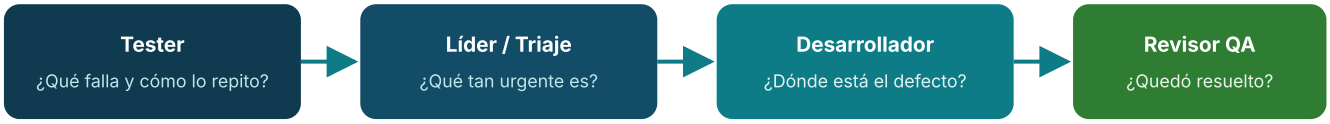
Término	Significado	Ejemplo en UMSSPIRA
Error (error)	Acción humana equivocada durante el desarrollo.	El desarrollador guarda el token de sesión en una cookie sin el atributo HttpOnly.
Defecto (defect, bug)	La manifestación del error en el código o el diseño.	La cookie umsspira_token queda legible desde JavaScript.
Falla (failure)	El efecto observable cuando el defecto se ejecuta.	document.cookie devuelve el token, que podría robarse vía XSS.
Bug report	El documento que registra la falla y permite llegar al defecto.	BUG-03 con pasos, evidencia y resultado esperado.

ATENCIÓN

El tester observa **fallas** y reporta **defectos**. No corresponde al reporte afirmar cuál fue el **error** humano ni la causa raíz exacta: eso lo determina quien corrige. El reporte describe lo observable y, a lo sumo, sugiere una hipótesis marcada como tal.

1.3 El reporte como acto de comunicación

Un reporte viaja entre personas que no comparten el mismo contexto: el tester que lo escribe, el líder que lo prioriza, el desarrollador que lo corrige y el revisor que lo verifica. Cada una lo lee con una pregunta distinta. El reporte debe contestarlas todas a la vez.



El mismo documento debe responder las cuatro preguntas sin aclaraciones externas.

Figura 1. Un reporte circula entre cuatro lectores con intereses distintos.

CAPÍTULO 2

2. Ciclo de vida de un defecto

El campo **Estado** del reporte se mueve a lo largo de un flujo. Conocer el flujo completo ayuda a redactar cada campo pensando en quién leerá el reporte en la siguiente etapa. El estado inicial que se escribe al crear el reporte es casi siempre "Nuevo".

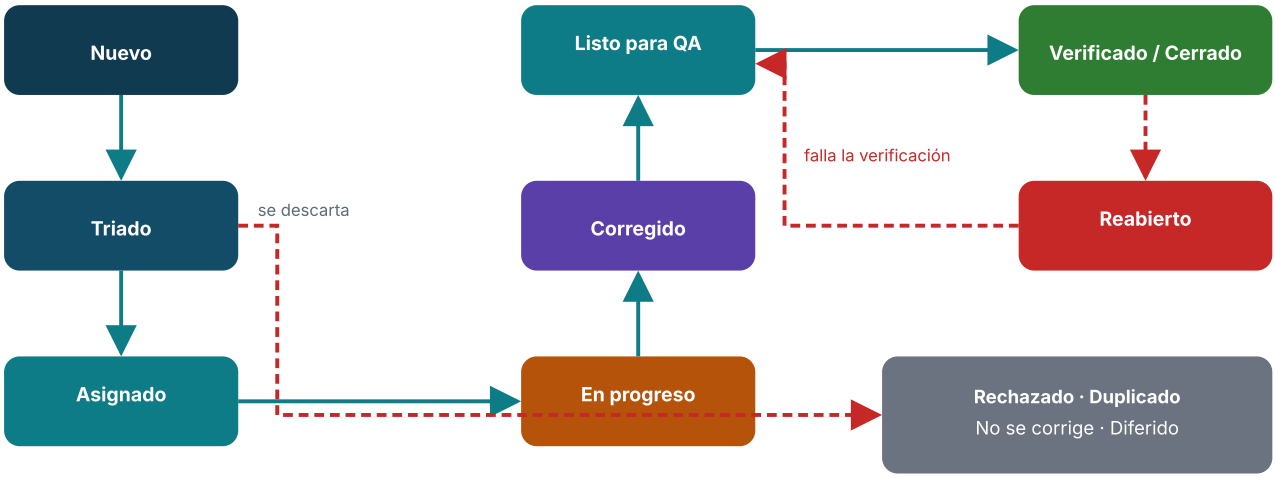


Figura 2. Flujo de estados. Las flechas rojas punteadas son retrocesos: un defecto puede reabrirse o descartarse.

Estado	Qué significa	Quién lo mueve
Nuevo	El reporte se creó y aún no fue revisado.	Tester que reporta.
Triado	Se confirmó, se fijó severidad y prioridad.	Líder de QA o comité de triaje.
Asignado	Tiene responsable de corrección.	Líder de desarrollo.
En progreso	Se está corrigiendo.	Desarrollador.
Corregido	El cambio está hecho, falta verificar.	Desarrollador.
Listo para QA	Desplegado en el ambiente de pruebas.	Desarrollador o despliegue.
Verificado / Cerrado	QA confirmó que el defecto desapareció.	Tester que verifica.
Reabierto	La verificación falló: el defecto sigue.	Tester que verifica.
Rechazado	No se reproduce o no es un defecto.	Triaje o desarrollo.
Duplicado	Ya existe otro reporte del mismo defecto.	Triaje.
No se corrige	Se decide convivir con el defecto.	Producto o triaje.
Diferido	Se corregirá en una versión posterior.	Producto o triaje.

Por qué importa al redactar

Un reporte termina en "Rechazado" o "Duplicado" casi siempre por mala redacción: no se reproduce con los pasos dados, el ambiente no coincide o ya existía otro reporte que una búsqueda previa habría encontrado. Escribir bien reduce esos cierres improductivos.

CAPÍTULO 3

3. Principios de un buen reporte

Seis principios definen la calidad de cualquier reporte. Las reglas de redacción del capítulo 5 son su aplicación concreta.

3.1 Reproducibilidad

El defecto se vuelve a producir siguiendo los pasos del reporte, en el ambiente indicado, sin conocimiento adicional. Si quien corrige no logra reproducirlo, el reporte se cierra como "No se reproduce" aunque el defecto exista. La reproducibilidad depende de tres cosas: pasos exactos, datos literales y ambiente identificado.

INCORRECTO

"A veces al guardar las áreas sale un error raro".

CORRECTO

"Con el token de sesión vencido, al abrir la sección Áreas técnicas y pulsar Reintentar, la petición se repite con el mismo token y vuelve a responder HTTP 401".

ATENCIÓN: defectos intermitentes

Si el defecto no se reproduce siempre, no se omite: se declara la frecuencia observada (por ejemplo "2 de 5 intentos") en el campo Reproducibilidad y se describe todo lo que se sabe del contexto en que aparece. Un defecto intermitente documentado vale más que uno silenciado.

3.2 Un defecto por reporte

Cada reporte describe una sola falla. Es el equivalente a la atomicidad de los casos de prueba. La prueba práctica: si para describir lo que falla se necesita la conjunción "y" entre dos fallas independientes, son dos reportes.

INCORRECTO

"El botón Guardar se ve raro y además el título de la pestaña no es descriptivo y el contraste de las etiquetas es bajo". Tres defectos en un reporte: no se pueden priorizar ni cerrar por separado.

CORRECTO

Tres reportes: BUG-01 (botón deshabilitado idéntico al habilitado), BUG-06 (título de pestaña no descriptivo) y BUG-08 (contraste insuficiente en etiquetas).

3.3 Objetividad

El reporte describe hechos observables y medibles, no juicios ni atribuciones de culpa. La objetividad protege el reporte de discusiones estériles y lo mantiene centrado en el defecto, no en las personas.

INCORRECTO

"El formulario está mal programado, cualquiera se habría dado cuenta de esto".

CORRECTO

"El formulario envía la petición de guardado aun cuando la lista de áreas está vacía; se esperaba el bloqueo del envío con el mensaje de validación".

3.4 Evidencia

Toda afirmación del reporte se respalda con una prueba que el lector puede inspeccionar: una captura anotada, un video corto, un fragmento de log, una respuesta HTTP, una consulta a la base de datos o una grabación de la consola del navegador. La evidencia convierte una afirmación en un hecho verificable.

3.5 Trazabilidad

El reporte cita la historia de usuario, el criterio de aceptación violado y el caso de prueba que detectó el defecto. Esa cadena permite responder "¿qué requisito incumple este defecto?" y medir cuánta de la funcionalidad quedó cubierta por las pruebas.



Figura 3. Cadena de trazabilidad: el defecto se ancla en el criterio de aceptación que incumple.

3.6 Accionabilidad

La suma de los cinco principios anteriores produce el sexto: un reporte accionable es aquel que quien corrige puede tomar y ejecutar sin pedir una sola aclaración. La accionabilidad es la medida última de calidad de un reporte, y es también el criterio con el que se evalúa esta guía.

CAPÍTULO 4

4. Anatomía de la plantilla

La plantilla estilo Qase que se usa en este trabajo agrupa los campos en bloques. El color de cada bloque ayuda a leerla de un vistazo: el encabezado identifica y clasifica, las precondiciones preparan, los pasos reproducen y los resultados contrastan. Esta es la estructura canónica; los capítulos siguientes explican cómo se llena cada celda.

ID	BUG-00	**Estado**	Nuevo
Prioridad	Alta / Media / Baja	**Severidad**	Bloqueante / Crítica / Mayor / Normal / Menor
Título	Qué falla, dónde y bajo qué condición		
Historia de Usuario	HU vinculada	**C.A. violado**	Criterio de aceptación incumplido
T.C. de origen	Caso de prueba que detectó el defecto	**Reproducibilidad**	Siempre (3/3) / Intermitente (n/m)
Ambiente	Versión, URL, navegador, resolución		
Descripción	Resumen del defecto y su impacto, en prosa breve		
Precondiciones	**Escenario:** estado del sistema antes de los pasos **Datos requeridos:** • datos sembrados o usuario necesario		
Nro	**Paso para reproducir**	**Datos de entrada**	
1	Acción única en infinitivo	`dato literal`	
2	Acción única en infinitivo		
Resultado actual	Lo que el sistema hace, observable y medible		
Resultado esperado	Lo que el sistema debería hacer según el C.A.		
Evidencias	Capturas, video, logs, respuesta HTTP		
Reportado por	Nombre y fecha		

Plantilla canónica. Los bloques coloreados: encabezado (azul), precondiciones (naranja claro), cabecera de pasos (violeta claro), resultado actual (rojo claro) y resultado esperado (verde claro).

REGLA

No se eliminan ni se agregan campos de la plantilla. Solo se agregan filas de pasos cuando hacen falta. Si un campo no aplica a un defecto concreto, su celda queda vacía, nunca con "N/A".

Bloque	Función	Capítulo
Encabezado (ID a Reproducibilidad)	Identifica, clasifica y prioriza el defecto.	5.1 a 5.6
Contexto (Ambiente, Descripción)	Sitúa el defecto y resume su impacto.	5.7 y 5.8
Preparación (Precondiciones)	Deja el sistema listo para reproducir.	5.9

Reproducción (Pasos, Datos)	Lleva al defecto paso a paso.	5.10 y 5.11
Contraste (Resultado actual y esperado)	Define el defecto por diferencia.	5.12 y 5.13
Respaldo (Evidencias, Reportado por)	Prueba la falla y da autoría.	5.14 y 5.15

CAPÍTULO 5

5. Redacción campo por campo

Este es el capítulo central. Para cada campo se indica qué contiene, cómo se redacta, el error típico y un par CORRECTO / INCORRECTO con material real de UMSSPIRA.

5.1 ID y Estado

El **ID** es el identificador único del reporte (por ejemplo BUG-03). Sirve para citarlo en reuniones, commits y matrices de trazabilidad. Se asigna de forma correlativa y no se reutiliza, aunque el reporte se cierre o se rechace. El **Estado** refleja el momento del ciclo de vida (capítulo 2); al crear el reporte casi siempre vale "Nuevo".

REGLA

El ID es inmutable y único. Un ID reutilizado rompe el historial: un commit que dice "corrige BUG-03" dejaría de tener sentido.

5.2 Prioridad y Severidad

Son dos ejes independientes y confundirlos es el error más común al clasificar un defecto. La **severidad** mide el impacto técnico del defecto sobre el sistema; la **prioridad** mide la urgencia con que el negocio quiere que se corrija. El capítulo 6 desarrolla la matriz completa; aquí basta la distinción.

Severidad (impacto)

La fija QA. Responde: ¿cuánto daña el defecto al sistema si ocurre? Escala: Bloqueante, Crítica, Mayor, Normal, Menor.

Prioridad (urgencia)

La fija el negocio o el líder. Responde: ¿qué tan pronto hay que corregirlo? Escala: Alta, Media, Baja.

INCORRECTO

"Severidad: urgente". Urgente no es severidad: mezcla los dos ejes y no pertenece a la escala.

CORRECTO

Token legible desde JavaScript: Severidad Mayor (riesgo de seguridad), Prioridad Alta (debe corregirse antes del despliegue).

ATENCIÓN

Un defecto puede ser de severidad baja y prioridad alta a la vez: por ejemplo, un error ortográfico en el logotipo de la portada es técnicamente menor, pero su visibilidad lo vuelve urgente. También al revés: una caída que solo ocurre en un navegador descontinuado es severa pero poco urgente.

5.3 Título

El título es la línea que más se lee y la que menos se cuida. Debe permitir entender el defecto sin abrir el detalle. Un buen título contiene tres elementos: **qué** falla, **dónde** ocurre y **bajo qué condición**. Se redacta en términos del comportamiento observable, no de la causa supuesta.

```
[ Qué falla ] + [ dónde ] + [ bajo qué condición ]
```

INCORRECTO

"Error en áreas". No dice qué error, ni dónde dentro de áreas, ni cuándo. Obliga a abrir el detalle para saber de qué trata.

CORRECTO

"El botón Guardar deshabilitado se ve igual que el habilitado en la sección Áreas técnicas". Qué, dónde y condición, todo en una línea.

Título débil	Título accionable
No guarda	El guardado de áreas no persiste tras recargar cuando se deselecciona la última área
Problema de sesión	La sesión vencida no se detecta y los datos del mentor siguen visibles hasta recargar
Se ve feo el contraste	Contraste insuficiente (2.9:1) en etiquetas naranjas sobre fondo claro, incumple WCAG 1.4.3

REGLA

El título describe el síntoma observable, no la causa. "El token se guarda en una cookie insegura" es una hipótesis de causa; el título correcto es "El token de sesión es legible desde JavaScript en la sección Áreas técnicas".

5.4 Historia de Usuario y C.A. violado

La **Historia de Usuario** ubica el defecto en la funcionalidad bajo prueba (por ejemplo HU-6.2 "Configurar áreas técnicas de especialidad"). El **C.A. violado** nombra el criterio de aceptación concreto que el defecto incumple. Este campo es el ancla de trazabilidad: convierte el defecto en el incumplimiento de un requisito acordado, no en una opinión del tester.

INCORRECTO

"C.A.: el que habla de guardar". Vago y no citable.

CORRECTO

"CA-FE-09: el sistema persiste las áreas seleccionadas y las muestra tras recargar". Cuando no hay un C.A. formal, se nombra el estándar o la heurística: "WCAG 1.4.3 Contraste mínimo" o "Consistencia de interfaz, heurística de Nielsen número 4".

5.5 T.C. de origen

Indica el caso de prueba cuya ejecución detectó el defecto (por ejemplo TC-108). Enlaza el defecto con la prueba que lo descubrió y permite, al cerrar el defecto, volver a ejecutar exactamente ese caso para verificarlo. Si el defecto surgió de pruebas exploratorias, se escribe "Pruebas exploratorias de UI" o "Revisión de código del cliente", según corresponda.

5.6 Reproducibilidad

Declara con qué frecuencia se reproduce el defecto siguiendo los pasos. Se expresa como una proporción de intentos: "Siempre (3/3)" cuando ocurre en todos los intentos, "Intermitente (2/5)" cuando no. La reproducibilidad orienta a quien corrige: un defecto intermitente suele deberse a condiciones de carrera, caché o dependencia de datos previos.

ATENCIÓN

Nunca se escribe "Siempre" sin haberlo intentado varias veces. La proporción se mide, no se supone. Un mínimo de tres intentos es una práctica razonable antes de declarar "Siempre".

5.7 Ambiente

Identifica dónde se observó el defecto con el detalle suficiente para reproducirlo en condiciones idénticas. Un defecto que solo aparece en un navegador o una resolución concretos es inreproducible si el ambiente no se documenta.

REGLA: qué incluir en el ambiente

- Ambiente lógico: desarrollo, QA, preproducción o producción.
- URL exacta del frontend y, si aplica, del backend o la API.
- Navegador y versión, o la aplicación y su versión.
- Sistema operativo y resolución de pantalla cuando el defecto es visual.
- Rol o tipo de usuario con el que se reprodujo.

CORRECTO

"QA en https://umsspira-web.vercel.app (API en https://umsspira-api-b48q.onrender.com); Google Chrome última versión estable; resolución 1366×768; rol mentor habilitado".

5.8 Descripción

Es un resumen en prosa breve del defecto y su impacto, pensado para quien prioriza y no quiere leer todos los pasos. Responde qué ocurre y por qué importa. No repite los pasos: los anticipa. Si el defecto tiene una hipótesis de causa basada en evidencia, va aquí marcada como hipótesis, nunca como hecho.

INCORRECTO

"Hay un bug". Sin contenido: no resume nada.

CORRECTO

"Ante respuestas con estado HTTP 500 o superior, la interfaz concatena y muestra al usuario el campo message devuelto por el backend. Esto puede exponer detalles internos del servidor. Observado por revisión del código del cliente; no reproducido en vivo".

5.9 Precondiciones

Describen el estado del sistema **antes** del primer paso. Se dividen en dos partes. El **Escenario** narra la situación de partida en una frase. Los **Datos requeridos** listan los usuarios, registros o configuraciones que deben existir para que los pasos tengan sentido. Unas precondiciones completas son lo que hace que el reporte sea independiente: quien lo ejecuta no necesita haber corrido ningún otro caso antes.

INCORRECTO

"Estar en la página". No dice con qué usuario, en qué estado ni con qué datos.

CORRECTO

Escenario: "Mentor habilitado en la sección Áreas técnicas con cuatro áreas guardadas y sin cambios pendientes". Datos requeridos: "Mentor de prueba con sesión iniciada en UMSSPIRA".

REGLA

Todo dato que los pasos usan debe estar sembrado en las precondiciones. Si el paso 3 marca el área "Ciencia de Datos", esa área debe existir en el catálogo según las precondiciones. Un paso que usa un dato no declarado es inreproducible.

5.10 Pasos para reproducir

Son la ruta mínima y ordenada que lleva al defecto desde el estado de las precondiciones. Cada paso es **una sola acción**, redactada con verbo en **infinitivo** (abrir, ingresar, seleccionar, pulsar, observar). El capítulo 7 detalla cómo construirlos y minimizarlos.

REGLA

Un paso, una acción. "Abrir la sección y marcar el área y pulsar Guardar" son tres pasos, no uno. Separarlos permite señalar con precisión en qué punto aparece el defecto.

INCORRECTO

"1. Hacer todo el flujo de guardado y ver que falla".

CORRECTO

"1. Iniciar sesión y abrir la sección Áreas técnicas. 2. Deseleccionar la última área marcada. 3. Pulsar Guardar. 4. Recargar la página".

ATENCIÓN

El último paso suele ser el que revela el defecto ("recargar", "observar el mensaje"). No se le adjunta el resultado en el mismo renglón: el resultado va en los campos Resultado actual y Resultado esperado, enfrentados.

5.11 Datos de entrada

Son los valores literales que un paso consume: un correo, un texto, una URL, el nombre de un área. Van en su columna, junto al paso que los usa. Deben ser exactos y reutilizables: quien reproduce copia el valor tal cual. Si un paso no consume ningún dato, la celda queda vacía.

INCORRECTO

Paso: "Ingresar un correo". Dato: "uno cualquiera".

CORRECTO

Paso: "Navegar a la sección Áreas técnicas". Dato: `https://umsspira-web.vercel.app/mentorship/profile/areas`

5.12 Resultado actual

Describe lo que el sistema **hace** al ejecutar los pasos, en términos observables y medibles. Es el síntoma, capturado con precisión: un texto exacto entre comillas, el estado de un control, un valor, un código HTTP, una fila en una tabla. Es la mitad del defecto.

INCORRECTO

"No funciona bien el guardado".

CORRECTO

"document.cookie devuelve umsspira_token (un JWT) y umsspira_role; el token es legible desde JavaScript y el atributo HttpOnly no está activado".

5.13 Resultado esperado

Describe lo que el sistema **debería** hacer según el criterio de aceptación o el estándar citado. Enfrentado al resultado actual, la diferencia entre ambos es, exactamente, el defecto. El resultado esperado nunca es "que funcione": es una afirmación verificable y anclada en el requisito.

INCORRECTO

"Que el token esté seguro".

CORRECTO

"El token de sesión reside en una cookie con los atributos HttpOnly y Secure, inaccesible desde JavaScript; el rol no se expone en una cookie legible y la autorización se valida con el token en el servidor".

REGLA

Resultado actual y resultado esperado describen la misma dimensión para que sean comparables. Si el actual habla del valor de document.cookie, el esperado habla de dónde debería residir el token, no de otra cosa.

5.14 Evidencias

Listan y describen las pruebas adjuntas que respaldan el reporte. No basta con "ver captura": se indica qué muestra cada evidencia y, cuando aplica, dónde mirar. El capítulo 9 detalla qué evidencia conviene a cada tipo de defecto.

INCORRECTO

"Adjunto imagen".

CORRECTO

"Captura de la consola con document.cookie mostrando el token; captura de Application > Cookies con los atributos de umsspira_token sin HttpOnly".

5.15 Reportado por

Registra el nombre de quien reporta y la fecha de creación. Da autoría y permite a quien corrige consultar dudas con la persona correcta. La fecha sitúa el defecto respecto de la versión del sistema en que se observó.

CAPÍTULO 6

6. Severidad y prioridad en profundidad

Clasificar bien un defecto es la mitad de un buen triaje. Severidad y prioridad son ejes ortogonales: se deciden por separado y se combinan al final.

6.1 Escala de severidad

La severidad la fija QA según el impacto técnico del defecto. Se decide con una pregunta: si el defecto ocurre, ¿cuánto daña al sistema o al usuario?

Severidad	Criterio	Ejemplo
Bloqueante	Impide continuar; no hay forma de evitarlo.	La página de áreas no carga: pantalla en blanco.
Crítica	Pérdida de datos, falla de seguridad grave o función principal inutilizable.	Guardar borra las áreas previas del mentor.
Mayor	Función importante falla, pero existe una forma de evitarlo.	La sesión vencida no se detecta hasta recargar.
Normal	Defecto funcional de alcance limitado.	El botón deshabilitado se ve igual que el habilitado.
Menor	Cosmético o de redacción, sin impacto funcional.	Falta de concordancia: "los 1 cambios realizados".

6.2 Escala de prioridad

La prioridad la fija el negocio o el líder según la urgencia de corrección, considerando visibilidad, cercanía de un despliegue y costo de convivir con el defecto.

Prioridad	Criterio
Alta	Debe corregirse antes del próximo despliegue o de inmediato.
Media	Se corrige en el ciclo actual, sin bloquear el despliegue.
Baja	Puede esperar a una versión posterior.

6.3 La matriz de decisión

Cruzar ambos ejes da una guía de atención. La severidad empuja desde la técnica; la prioridad, desde el negocio. La esquina superior izquierda se atiende primero; la inferior derecha, al final.

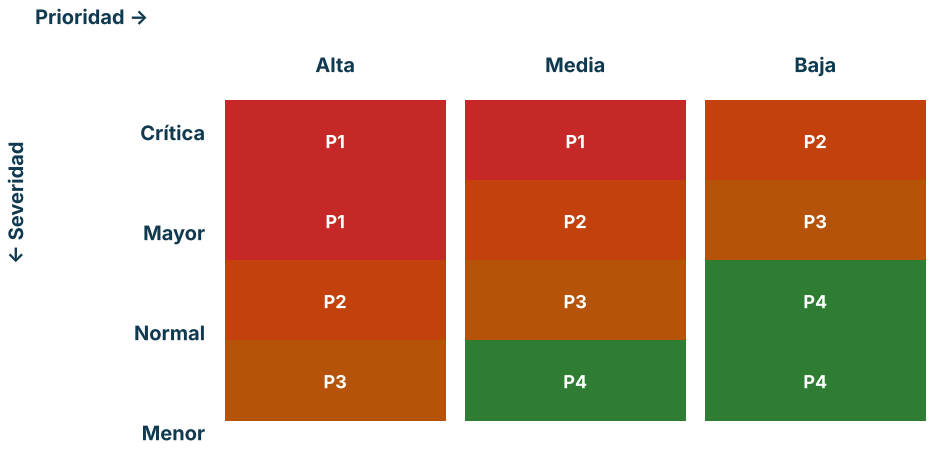


Figura 4. Matriz severidad por prioridad. P1 se atiende primero; P4, al final. Bloqueante se trata como Crítica o por encima.

ATENCIÓN: el caso de seguridad

Un defecto de seguridad eleva su prioridad por encima de lo que sugeriría su severidad funcional aislada, porque el costo de explotación es alto aunque la función siga operando. El token legible desde JavaScript no rompe ninguna función, y aun así es Prioridad Alta.

CAPÍTULO 7

7. Cómo construir los pasos de reproducción

Los pasos son la parte del reporte que más trabajo ahorra o más tiempo hace perder. El objetivo es la ruta más corta y determinista que lleva al defecto.

7.1 El procedimiento

1. Reproducir el defecto una vez sin anotar, para entender el camino.
2. Reproducirlo de nuevo anotando cada acción como un paso de una sola operación.
3. Eliminar los pasos que no influyen en la aparición del defecto.
4. Reproducirlo una tercera vez siguiendo solo los pasos anotados, como lo haría un extraño.
5. Anotar la frecuencia observada en el campo Reproducibilidad.

7.2 Minimización

Un buen reporte no describe todo lo que se hizo, sino lo mínimo necesario para llegar al defecto. Cada paso sobrante es una oportunidad de que quien reproduce se desvíe. La prueba de minimización: si quitar un paso no impide que el defecto aparezca, ese paso sobra.

INCORRECTO (nueve pasos, siete sobran)

Iniciar sesión, abrir el menú, leer la bienvenida, ir al perfil, revisar los datos, volver al inicio, abrir mentorías, abrir áreas, y entonces reproducir.

CORRECTO (ruta mínima)

Iniciar sesión, abrir la sección Áreas técnicas, reproducir el defecto. Los pasos intermedios no influyen en su aparición.

7.3 Verbos y redacción

Cada paso empieza con un verbo en infinitivo que nombra una acción observable del usuario. Se nombran botones, campos, enlaces y mensajes por su texto visible, entre comillas cuando ayuda a ubicarlos.

Verbos de acción (hacer)	Verbos de verificación (observar)
Iniciar, abrir, navegar, ingresar, seleccionar, marcar, deseleccionar, pulsar, recargar, esperar.	Observar, leer, comparar, revisar, confirmar, contar.

REGLA

El paso describe la acción del usuario, no la respuesta del sistema. "Pulsar Guardar" es un paso; "el sistema guarda" no es un paso, es un resultado. La respuesta del sistema vive en Resultado actual.

7.4 Pasos de alto nivel

Los pasos describen lo que el usuario hace y observa, no la implementación. Se nombran controles y mensajes visibles y códigos HTTP; no se nombran clases, servicios internos ni consultas. La excepción es un defecto de capa API o de revisión de código, donde el actor es el tester inspeccionando la respuesta o el código y lo observable es la respuesta HTTP, el valor de una cookie o un fragmento del cliente.

CAPÍTULO 8

8. Clasificación de defectos por tipo

El tipo de defecto orienta qué evidencia conviene, qué criterio se cita y qué severidad suele corresponder. Un buen tester reconoce los ocho tipos y los busca de forma deliberada.

Tipo	Qué se busca	Ejemplo en UMSSPIRA
Funcional	El sistema no hace lo que el requisito exige.	El guardado no persiste tras recargar.
Interfaz / cosmético	Aspecto visual, alineación, color, texto, concordancia.	El botón deshabilitado se ve igual que el habilitado.
Seguridad	Exposición de datos, control de acceso, sesión, inyección.	Token de sesión legible desde JavaScript (riesgo XSS).
Accesibilidad	Incumplimiento de WCAG: contraste, foco, títulos, etiquetas.	Contraste insuficiente en etiquetas naranjas (WCAG 1.4.3).
Rendimiento	Lentitud, consumo excesivo, tiempos de respuesta altos.	La carga del catálogo tarda más del umbral acordado.
Compatibilidad	Falla en un navegador, dispositivo o resolución concretos.	El diseño se rompe en pantallas menores a 360px.
Usabilidad	El flujo confunde o induce error, aunque funcione.	Reintentar repite el mismo token vencido sin resolver nada.
Datos / integridad	Datos inconsistentes, duplicados o mal validados.	Se permite guardar con la lista de áreas vacía.

Nombrar el criterio según el tipo

Cada tipo se ancla en una referencia distinta: los funcionales y de datos citan un criterio de aceptación de la HU; los de accesibilidad citan una pauta WCAG; los de usabilidad citan una heurística de Nielsen; los de seguridad citan la práctica incumplida (por ejemplo cookie sin HttpOnly, o exposición de mensajes de error del servidor).

ATENCIÓN: límites de las pruebas de seguridad

Observar un riesgo de seguridad desde el cliente (una cookie legible, un mensaje de error que filtra detalle interno) es parte del trabajo de QA. Ejecutar una intrusión real contra un backend desplegado (peticiones autenticadas manipuladas, intentos de acceder a datos de otro usuario) no lo es sin autorización explícita. Cuando el defecto es de seguridad y no se ejecutó una explotación, se declara así en el reporte: "observado por revisión del lado del cliente; pendiente de verificación del lado del servidor; no se ejecutó ninguna explotación".

CAPÍTULO 9

9. Evidencias efectivas

La evidencia convierte una afirmación en un hecho. Un reporte sin evidencia obliga a quien corrige a reproducir desde cero solo para creer el síntoma.

9.1 Qué evidencia conviene a cada tipo

Tipo de defecto	Evidencia recomendada
Interfaz / cosmético	Captura anotada; valores computados de estilo (color, contraste, tamaño) desde las herramientas del navegador.
Funcional	Video corto del flujo; captura del estado final; datos antes y después en la base.
Seguridad	Captura de la consola o del panel de almacenamiento; respuesta HTTP; fragmento del código del cliente.
Accesibilidad	Reporte de una herramienta de auditoría; medición del contraste; captura del foco del teclado.
Rendimiento	Captura del panel de red con tiempos; traza de rendimiento; archivo HAR.
Compatibilidad	Captura en el navegador o la resolución donde falla, junto a la que funciona.

9.2 Cómo anotar una captura

Una captura sin anotar obliga a buscar el defecto dentro de la imagen. La captura útil señala el punto exacto con un recuadro o una flecha y, cuando aplica, muestra el valor medido. La siguiente captura real de la pantalla de áreas de UMSSPIRA concentra tres defectos distintos; una evidencia bien anotada marcaría cada uno.

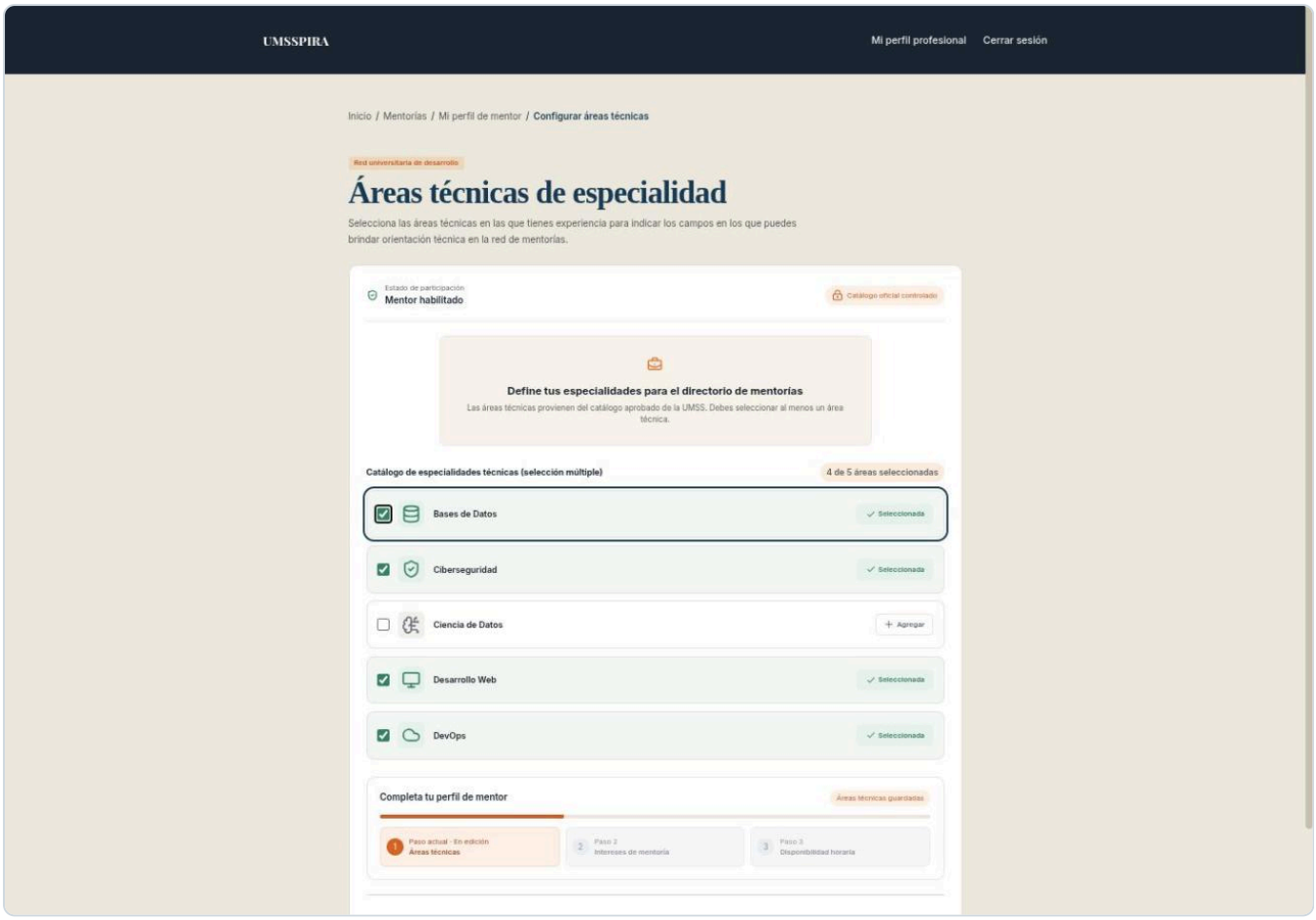


Figura 5. Pantalla real "Áreas técnicas de especialidad" de UMSSPIRA. Concentra material de varios reportes: el contador y las etiquetas naranjas (contraste, BUG-08), los tamaños de fuente reducidos (BUG-09) y, más abajo, el botón Guardar (BUG-01).

REGLA

La evidencia acompaña, no sustituye. Un video no reemplaza los pasos escritos: quien corrige necesita poder copiar un dato o saltar a un paso sin ver el video entero. La evidencia prueba; los pasos reproducen.

9.3 Datos sensibles en la evidencia

ATENCIÓN

Antes de adjuntar una captura o un log, se ocultan credenciales, tokens completos, datos personales y cualquier secreto. Para ilustrar un token legible basta mostrar que existe y es accesible, nunca su valor completo. Un reporte es un documento que circula: no debe filtrar lo mismo que denuncia.

CAPÍTULO 10

10. Ejemplos completos resueltos

Tres reportes llenos de principio a fin. Los dos primeros son defectos reales de UMSSPIRA; el tercero es un caso genérico de comercio electrónico.

Ejemplo A: defecto de seguridad (real, UMSSPIRA)

ID	BUG-03	**Estado**	Nuevo
Prioridad	Alta	**Severidad**	Mayor
Título	El token de sesión (JWT) es legible desde JavaScript por una cookie sin HttpOnly en la sección Áreas técnicas		
Historia de Usuario	HU-6.2 Configurar áreas técnicas de especialidad	**C.A. violado**	Seguridad de sesión: el token no debe ser accesible desde el cliente
T.C. de origen	Pruebas exploratorias de seguridad	**Reproducibilidad**	Siempre (3/3)
Ambiente	QA en https://umsspira-web.vercel.app; Google Chrome última versión estable; rol mentor habilitado		
Descripción	El token de sesión se almacena en una cookie accesible desde JavaScript y se envía como Bearer. Un ataque XSS podría exfiltrar el token y suplantar la sesión. No se ejecutó ninguna explotación; pendiente de verificación del lado del servidor.		
Precondiciones	**Escenario:** Usuario con sesión iniciada en UMSSPIRA con rol de mentor. **Datos requeridos:** • Mentor de prueba con sesión iniciada en UMSSPIRA.		

Nro	Paso para reproducir	Datos de entrada
1	Iniciar sesión en UMSSPIRA.	`credenciales del mentor de prueba`
2	Abrir las herramientas de desarrollo y la pestaña Console.	
3	Leer el contenido de las cookies en la consola.	`document.cookie`
4	Revisar en Application > Cookies el atributo HttpOnly de la cookie del token.	`umsspira_token`

Resultado actual	document.cookie devuelve umsspira_token (JWT) y umsspira_role; el token es legible desde JavaScript y el atributo HttpOnly no está activado.
Resultado esperado	El token reside en una cookie con HttpOnly y Secure, inaccesible desde JavaScript; el rol no se expone en una cookie legible y la autorización se valida con el token en el servidor.
Evidencias	Captura de la consola con document.cookie y captura de Application > Cookies con los atributos de umsspira_token (valor del token oculto).
Reportado por	Pardo Pozo Juan Diego · 12/10/2026

Ejemplo B: defecto cosmético (real, UMSSPIRA)

ID	BUG-01	**Estado**	Nuevo
Prioridad	Media	**Severidad**	Normal
Título	El botón Guardar deshabilitado se ve igual que el habilitado en la sección Áreas técnicas		

Historia de Usuario	HU-6.2 Configurar áreas técnicas de especialidad	**C.A. violado**	CA-NF-13 (Accesibilidad, WCAG 2.1 AA)
T.C. de origen	Pruebas exploratorias de UI	**Reproducibilidad**	Siempre (3/3)
Ambiente	QA en https://umsspira-web.vercel.app; Google Chrome última versión estable; resolución 1366×768		
Descripción	El botón Guardar no cambia de apariencia entre los estados habilitado y deshabilitado: ambos usan el mismo fondo naranja y opacidad 1. La única diferencia es el cursor not-allowed al pasar el puntero, imperceptible sin mouse. El usuario no distingue cuándo puede guardar.		
Precondiciones	**Escenario:** Mentor habilitado en la sección Áreas técnicas con cuatro áreas guardadas y sin cambios pendientes. **Datos requeridos:** • Mentor de prueba con sesión iniciada en UMSSPIRA.		

Nro	Paso para reproducir	Datos de entrada
1	Abrir Google Chrome y navegar a la sección Áreas técnicas.	`https://umsspira-web.vercel.app/mentorship/profile/areas`
2	Observar el aspecto del botón Guardar sin cambios pendientes.	
3	Marcar una casilla de área no seleccionada del catálogo.	`Ciencia de Datos`
4	Observar el aspecto del botón Guardar con un cambio pendiente.	
5	Comparar el color de fondo y la opacidad del botón en ambos estados.	

Resultado actual	Con y sin cambios el botón Guardar tiene el mismo fondo naranja (rgb 255,182,107) y opacidad 1; solo cambia el cursor a not-allowed. No hay indicación visual del estado deshabilitado.
Resultado esperado	El botón deshabilitado se distingue visualmente del habilitado (menor opacidad o color atenuado) y mantiene contraste suficiente, conforme a WCAG 2.1 AA.
Evidencias	Captura del botón en ambos estados y valores computados de background-color y opacity.
Reportado por	Pardo Pozo Juan Diego · 12/10/2026

Ejemplo C: defecto funcional (genérico, comercio electrónico)

ID	BUG-114	**Estado**	Nuevo
Prioridad	Alta	**Severidad**	Crítica
Título	El total del carrito no descuenta el cupón al cambiar la cantidad de un producto		
Historia de Usuario	HU-4.1 Aplicar cupones de descuento en el carrito	**C.A. violado**	CA-FE-02: el descuento se recalcula ante cualquier cambio del carrito
T.C. de origen	TC-220 Recálculo del descuento	**Reproducibilidad**	Siempre (3/3)
Ambiente	QA en https://tienda.qa.local; Google Chrome última versión estable; usuario comprador estándar		
Descripción	Al aumentar la cantidad de un producto después de aplicar un cupón porcentual, el subtotal se actualiza pero el descuento conserva el valor calculado sobre la cantidad anterior, por lo que el total cobrado es mayor al correcto.		

Precondiciones	**Escenario:** Comprador con un producto en el carrito y un cupón porcentual aplicado. **Datos requeridos:** • Usuario comprador; producto a 100 unidades monetarias; cupón DESC10 del 10%.	
Nro	**Paso para reproducir**	**Datos de entrada**
1	Iniciar sesión y abrir el carrito con un producto.	`cantidad 1, precio 100`
2	Aplicar el cupón porcentual.	`DESC10`
3	Observar el total con el descuento aplicado.	
4	Aumentar la cantidad del producto a dos unidades.	`cantidad 2`
5	Observar el subtotal, el descuento y el total.	
Resultado actual	El subtotal pasa a 200 pero el descuento sigue en 10 (10% de 100); el total queda en 190 en vez de 180.	
Resultado esperado	El descuento se recalcula al cambiar la cantidad: 10% de 200 es 20; el total queda en 180.	
Evidencias	Video del recálculo; captura del desglose del carrito con subtotal, descuento y total tras el paso 5.	
Reportado por	Pardo Pozo Juan Diego · 12/10/2026	

CAPÍTULO 11

11. Lenguaje: palabras y estilo

El reporte se redacta en un registro técnico, neutro y preciso. El estilo no es adorno: palabras vagas producen reportes inreproducibles.

11.1 Vocabulario

Evitar	Preferir	Por qué
"funciona mal", "anda raro"	el síntoma exacto observado	Lo vago no se puede verificar.
"a veces"	la frecuencia medida (2/5)	Cuantifica la intermitencia.
"debería funcionar"	el resultado esperado según el C.A.	Ancla la expectativa en el requisito.
"obviamente", "es evidente"	se omiten	Si fuera obvio, no habría defecto.
"el programador no..."	el sistema hace o no hace X	El reporte no juzga personas.

REGLA

No se usa "correctamente" ni "adecuadamente": no son verificables. En su lugar se escribe qué debería ocurrir de forma concreta: "muestra el mensaje de confirmación", no "guarda correctamente".

11.2 Tiempo verbal y voz

Los pasos van en infinitivo (abrir, pulsar, observar). El resultado actual va en presente del indicativo y describe lo que el sistema hace (devuelve, muestra, permite). El resultado esperado va en el modo que exprese la expectativa (debería mostrar, reside, se recalcula). Se prefiere la voz activa: el sujeto es el sistema o el usuario, no una abstracción.

11.3 Precisión numérica y textual

Los textos de la interfaz se citan entre comillas y tal como aparecen, con sus mayúsculas y su puntuación. Los valores numéricos se escriben con su unidad (12px, 2.9:1, HTTP 401). Un código de error se escribe tal cual lo devuelve el sistema. La precisión aquí es la diferencia entre un reporte que se reproduce y uno que se descarta.

CAPÍTULO 12

12. Antipatrones frecuentes

Estos son los errores que más veces convierten un reporte válido en uno que se rechaza o se ignora. Reconocerlos al releer evita casi todos los cierres improductivos.

Antipatrón	En qué consiste	Corrección
Reporte multifalla	Varios defectos en un solo reporte unidos por "y".	Un reporte por defecto.
Título genérico	"Error en el formulario" sin qué, dónde ni condición.	Título con los tres elementos.
Pasos con huecos	Saltar acciones que el autor da por sabidas.	Reproducir como un extraño.
Resultado sin esperado	Decir qué falla sin decir qué debería ocurrir.	Enfrentar actual y esperado.
Diagnóstico como hecho	Afirmar la causa raíz sin evidencia.	Hipótesis marcada como tal, o se omite.
Juicio de valor	Calificar el trabajo o a la persona.	Hechos observables.
Ambiente ausente	No decir navegador, versión ni URL.	Ambiente completo.
Evidencia muda	Adjuntar una captura sin señalar qué muestra.	Anotar y describir la evidencia.
"N/A" por todas partes	Rellenar campos vacíos con ruido.	Dejar la celda vacía.
Severidad igual a prioridad	Confundir impacto con urgencia.	Decidir cada eje por separado.

CAPÍTULO 13

13. Lista de verificación final

Antes de cambiar el estado a "Nuevo" y enviar el reporte, se recorre esta lista. Si algún punto falla, el reporte aún no está listo.

Identificación y clasificación

- ☐ El ID es único y no reutilizado.
- ☐ El título dice qué falla, dónde y bajo qué condición, sin nombrar la causa.
- ☐ La severidad refleja el impacto técnico y la prioridad la urgencia, decididas por separado.

Trazabilidad y contexto

- ☐ El reporte cita la HU, el C.A. o estándar violado y el caso de prueba de origen.
- ☐ El ambiente incluye versión, URL, navegador, resolución y rol cuando aplican.
- ☐ La descripción resume el defecto y su impacto sin repetir los pasos.

Reproducción

- ☐ Las precondiciones dejan el sistema listo: escenario y datos requeridos completos.
- ☐ Cada paso es una sola acción en infinitivo y no hay pasos sobrantes.
- ☐ Todo dato que un paso usa está declarado y es literal.
- ☐ La frecuencia de reproducción está medida, no supuesta.

Contraste y respaldo

- ☐ Resultado actual y esperado describen la misma dimensión y son verificables.
- ☐ El resultado esperado se ancla en el criterio citado, no en "que funcione".
- ☐ Cada afirmación tiene evidencia, anotada y sin datos sensibles expuestos.
- ☐ El reporte describe un solo defecto.

> **La prueba del extraño**
>
> La verificación última: entregar el reporte a alguien que no vio el defecto y pedirle que lo reproduzca siguiendo solo lo escrito. Si lo logra sin preguntar nada, el reporte está listo. Si pregunta algo, esa pregunta señala el campo que falta completar.

CAPÍTULO 14

14. Glosario

Siglas y términos técnicos usados en esta guía, expandidos y definidos.

Término	Definición
Bug report	Reporte formal y reproducible de un defecto del software.
Defecto (defect, bug)	Discrepancia entre el comportamiento observado y el esperado, originada en el código o el diseño.
Error (error)	Acción humana equivocada que introduce un defecto.
Falla (failure)	Efecto observable cuando un defecto se ejecuta.
Severidad	Medida del impacto técnico de un defecto sobre el sistema.
Prioridad	Medida de la urgencia con que el negocio quiere corregir un defecto.
Triaje (triage)	Proceso de revisar, confirmar, clasificar y priorizar los defectos reportados.
Reproducibilidad	Propiedad de un defecto de volver a producirse siguiendo los mismos pasos; se expresa como proporción de intentos.
Atomicidad	Propiedad de describir un solo defecto o comportamiento por artefacto.
Precondición	Estado y datos que deben existir antes de ejecutar los pasos.
Resultado actual	Lo que el sistema hace al ejecutar los pasos, observable y medible.
Resultado esperado	Lo que el sistema debería hacer según el criterio de aceptación.
HU (Historia de Usuario)	Descripción de una funcionalidad desde la perspectiva de la persona usuaria.
C.A. (Criterio de Aceptación)	Condición que debe cumplirse para dar por válida una historia de usuario.
T.C. (Test Case, caso de prueba)	Conjunto de pasos y datos para verificar un comportamiento esperado.
Épica (epic)	Agrupación de historias de usuario relacionadas en una funcionalidad mayor.
Qase	Herramienta de gestión de pruebas cuyo formato de campos inspira la plantilla usada.
JWT (JSON Web Token)	Formato de token firmado que transporta la identidad y los permisos de la sesión.
HttpOnly	Atributo de una cookie que impide que JavaScript la lea; protege el token.
Secure	Atributo de cookie que la envía solo por conexiones cifradas (HTTPS).
XSS (Cross-Site Scripting)	Vulnerabilidad que permite inyectar y ejecutar código malicioso en el navegador de la víctima.
HTTP (HyperText Transfer Protocol)	Protocolo de comunicación entre el navegador y el servidor.
HTTP 401	Código que indica que la petición carece de credenciales válidas o la sesión venció.
HTTP 5xx	Familia de códigos que indican un error del lado del servidor.
HAR (HTTP Archive)	Archivo que registra todas las peticiones de red de una sesión del navegador.

WCAG (Web Content Accessibility Guidelines)	Pautas internacionales de accesibilidad para contenido web.
WCAG 1.4.3	Pauta de contraste mínimo: al menos 4.5:1 para texto normal y 3:1 para texto grande.
Heurística de Nielsen	Conjunto de diez principios de usabilidad para evaluar interfaces.
Egresado	Persona que concluyó el plan de estudios de una carrera; usuario objetivo de la plataforma de mentorías.

Cierre

Un bug report bien escrito se reconoce por una sola cosa: quien lo recibe puede actuar sin preguntar nada. Todo en esta guía sirve a ese fin. Cuando dudes de un campo, vuelve a la prueba del extraño del capítulo 13.